



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

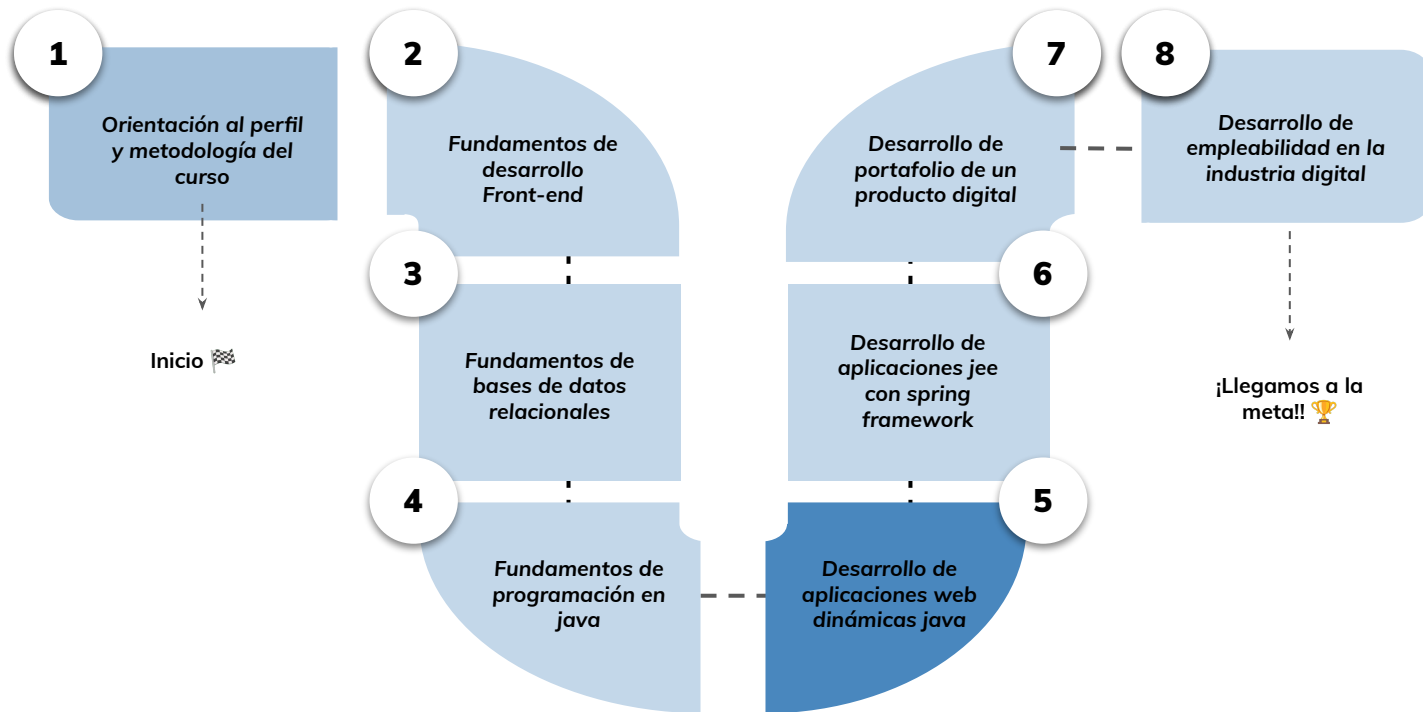


› Capa de acceso a datos - Parte II

Aprendizaje Esperado 4: Implementar capa de acceso a datos en una aplicación web dinámica utilizando el patrón dao para resolver un problema determinado.

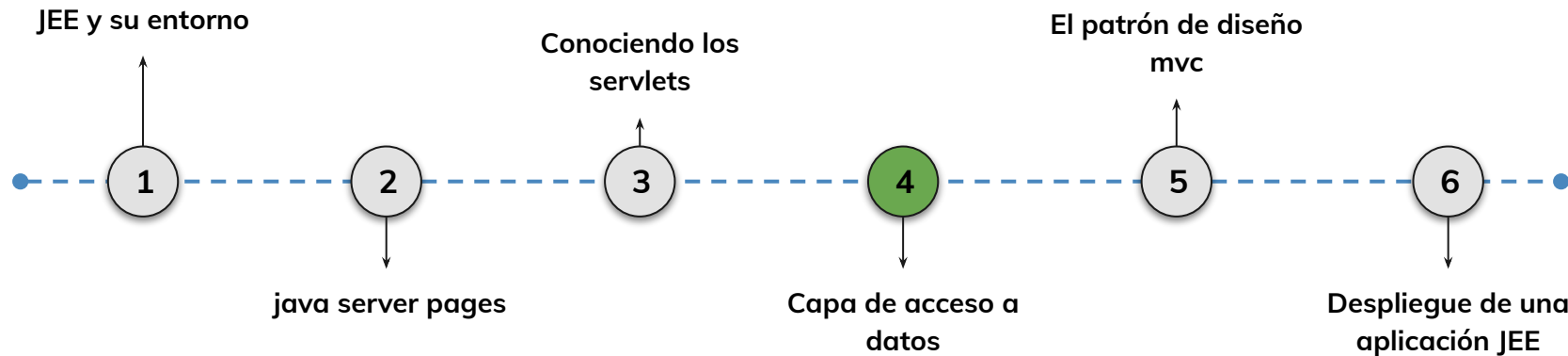
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



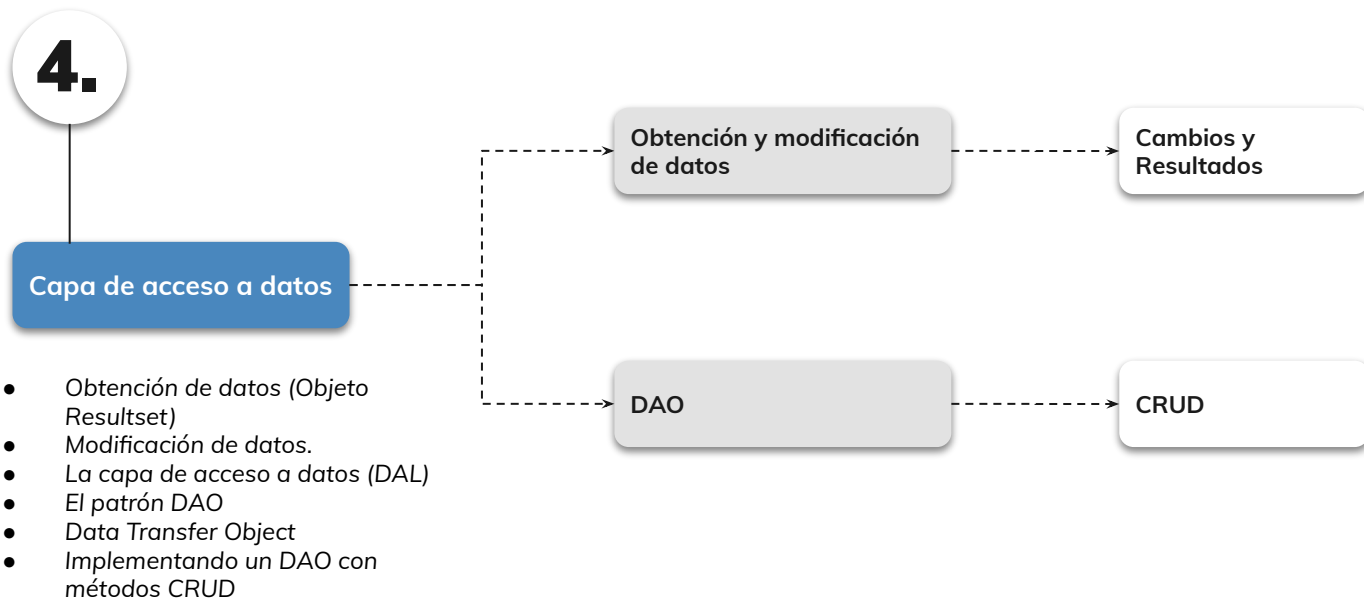
Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?



Objetivos de aprendizaje

¿Qué aprenderás?

- Obtener datos con Resultset
- Modificar datos con JDBC
- Conocer la capa de acceso a datos (DAL)
- Implementar una clase DAO para interactuar con la base de datos
- Conocer el patrón DTO

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Conocer el acceso a datos desde la aplicación
- Aprender a configurar JDBC para conectarse a una base de datos
- Comprender la implementación del patrón de diseño Singleton para la conexión.
- Aprender la implementación del objeto Statement.

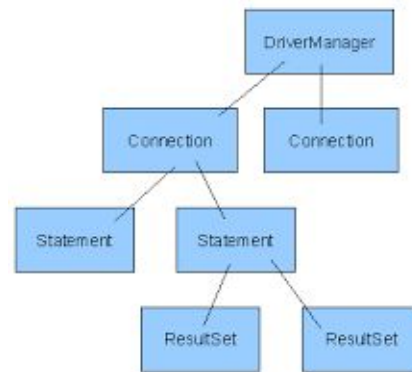
Obtención de datos:

ResultSet

ResultSet

Para obtener datos almacenados en la BD se utiliza una consulta SQL (query). La consulta se puede ejecutar utilizando el objeto **Statement**, con el método **executeQuery()** al que se le pasa una cadena con la consulta SQL. **Los datos resultantes se devuelven como un objeto ResultSet**.

```
ResultSet result = stmt.executeQuery(sentenciaSQL);
```



ResultSet

Para obtener datos almacenados en la BD se utiliza una consulta SQL (query). La consulta se puede ejecutar utilizando el objeto **Statement**, con el método **executeQuery()** al que se le pasa una cadena con la consulta SQL. **Los datos resultantes se devuelven como un objeto ResultSet** .

```
ResultSet rs = stmt.executeQuery(sentenciaSQL);
```

```
// Crear sentencia SQL
String sql = "SELECT * FROM products";

// Preparar sentencia
Statement stmt = conn.createStatement();

// Ejecutar consulta
ResultSet rs = stmt.executeQuery(sql);
```

ResultSet

La consulta SQL devolverá una tabla que tendrá una serie de campos y un conjunto de registros, cada uno de los cuales consistirá en una tupla de valores correspondientes a los campos de la tabla.

El objeto **ResultSet** proporciona el acceso a los datos de estas filas mediante un **conjunto de métodos get** que permiten el acceso a las diferentes columnas de las filas.

El método **ResultSet.next** se usa para moverse a la siguiente fila del ResultSet, convirtiendo a ésta en la fila actual.

ResultSet

La consulta SQL devolverá una tabla que tendrá una serie de campos y un conjunto de registros, cada uno de los cuales consistirá en una tupla de valores correspondientes a los campos de la tabla.

El objeto **ResultSet** proporciona el acceso a los datos de estas filas mediante un **conjunto de métodos get** que permiten el acceso a las diferentes columnas de las filas.

El método **ResultSet.next** se usa para moverse a la siguiente fila del ResultSet, convirtiendo a ésta en la fila actual.

```
(rs.getString("nombre"))  
(rs.getDouble("precio"))
```

ResultSet

El formato general de un ResultSet es una tabla con cabeceras de columna y los valores correspondientes devueltos por la “query”.

Por ejemplo, si la “query” es `SELECT a, b, c FROM Table1`, el resultado tendrá una forma semejante a:

a	b	c
12345	Chile	10,5
31245	Brasil	22,7
47899	Perú	56,7

ResultSet

El siguiente fragmento de código es un ejemplo de la ejecución de una sentencia SQL que devolverá una colección de filas, con la columna 1 como un int, la columna 2 como una String y la columna 3 como un valor real:

a	b	c
12345	Chile	10,5
31245	Brasil	22,7
47899	Perú	56,7

```
Statement stmt = connection.createStatement();
ResultSet r = stmt.executeQuery("SELECT a, b, c FROM Table1");
while (r.next()) {
    int i = r.getInt("a");
    String s = r.getString("b");
    double d = r.getDouble("c");
    // imprimimos los valores de la fila actual
    System.out.println("Fila = " + i + " " + s + " " + d);
}
```

ResultSet

Filas ResultSet

Un ResultSet mantiene un cursor que apunta a la fila actual de datos. El cursor se mueve **una fila hacia abajo cada vez que se llama al método .next**.

Las filas de ResultSet se recuperan en secuencia desde la fila más alta a la más baja.

```
while(rs.next()) {  
    Product p = new Product();  
    p.setId(rs.getInt("id"));  
    p.setNombre(rs.getString("nombre"));  
    p.setPrecio(rs.getDouble("precio"));  
}
```

ResultSet

Columnas ResultSet

Los métodos **getX** suministran los medios **para recuperar los valores de las columnas** de la fila actual. Dentro de cada fila, los valores de las columnas pueden recuperarse en cualquier orden, pero para asegurar la máxima portabilidad, deberían extraerse las columnas de izquierda a derecha y leer los valores de las columnas una única vez.

```
while(rs.next()) {  
    Product p = new Product();  
    p.setId(rs.getInt("id"));  
    p.setNombre(rs.getString("nombre"));  
    p.setPrecio(rs.getDouble("precio"));  
}
```


ResultSet

Puede usarse, o bien el nombre de la columna o el número de columna, para referirse a esta . Por ejemplo: si la columna segunda de un objeto ResultSet rs se denomina “nombre” y almacena valores de cadena, cualquiera de los dos ejemplos siguientes nos devolverá el valor almacenado en la columna.

```
String s = rs.getString("nombre");  
String s = rs.getString(2);
```

```
while(rs.next()) {  
    Product p = new Product();  
    p.setId(rs.getInt("id"));  
    p.setNombre(rs.getString("nombre"));  
    p.setPrecio(rs.getDouble("precio"));  
}
```

ResultSet

Las columnas **se enumeran de izquierda a derecha** comenzando con la columna 1. Además, **los nombres** usados como input en los métodos getX **son insensibles a las mayúsculas**.

Algunos de los datos que podemos traer con el método get son:

getInt()	Sirve para obtener un numero entero de la base de datos
getLong()	Sirve para obtener un número long de la base de datos
getDouble()	Sirve para obtener un número real de la base de datos
getBoolean()	Sirve para obtener un booleano de la base de datos
getString()	Sirve para obtener una cadena de la base de datos
getDate()	Sirve para obtener una fecha de la base de datos

LIVE CODING

¡Utilizando ResultSet!

Vamos a trabajar en la clase que creamos con la conexión Singleton. Esta clase tiene métodos para conectarse, desconectarse y realizar una consulta (sentencia).

Ahora vamos a agregar el objeto ResultSet para que nos devuelva los resultados de la sentencia.

Tiempo: 20 minutos

Modificación de datos

< > Modificación de datos

El objeto **Statement** también posee el método **executeUpdate** : recibe como parámetro la cadena de caracteres que contiene la sentencia SQL a ejecutar.

Este método **únicamente permite realizar sentencias de actualización** de la BD: creación de tablas (**CREATE**), inserción (**INSERT**) y borrado de datos (**DELETE**). El método a utilizar es el siguiente:

```
stmt.executeUpdate(sentenciaSql);
```



< > Modificación de datos

Vamos a ver a continuación un ejemplo de estas operaciones.

Crearemos una tabla ALUMNOS en nuestra base de datos y añadiremos datos a la misma. La sentencia para la creación de la tabla será la siguiente:

```
// Creamos una tabla llamada Alumnos
String st_create = "CREATE TABLE ALUMNOS ("
    + " expediente INTEGER,"
    + " nombre VARCHAR(32),"
    + "PRIMARY KEY (expediente)"
    + ")";

stmt.executeUpdate(st_create);
```



< > Modificación de datos

Una vez creada la tabla podremos insertar datos en ella:

```
// Insertamos datos
String st_inserta = "INSERT INTO ALUMNOS(expediente, nombre)"
                  + "VALUES(1285, 'Manu')";

stmt.executeUpdate(st_inserta);
```



< > Modificación de datos

Cuando tengamos datos dentro de la tabla, podremos modificarlos utilizando para ello una sentencia **UPDATE**:

```
// Modificamos el registro
String st_actualiza = "UPDATE FROM ALUMNOS"
                    + "SET nombre = 'Nadu'"
                    + "WHERE expediente = 1285";

stmt.executeUpdate(st_actualiza);
```



< > Modificación de datos

Si queremos eliminar un registro de la tabla utilizaremos una sentencia DELETE:

```
// Borramos el registro
String st_borra = "DELETE FROM ALUMNOS"
                + "WHERE expediente = 1285";

stmt.executeUpdate(st_borra);
```

El método **executeUpdate** **retorna un entero** que nos dice el número de registros a los que ha afectado la operación, en caso de sentencias INSERT, UPDATE y DELETE. La creación de tablas retorna siempre 0.



LIVE CODING

¡Modificando Registros!

- Vamos a crear una nueva tabla en la DB llamada “Cuenta”.
 - ↪ Esta tabla tendrá las columnas: id, saldo y fecha.
- Luego, vamos a modificar el saldo.

Tiempo: 20 minutos

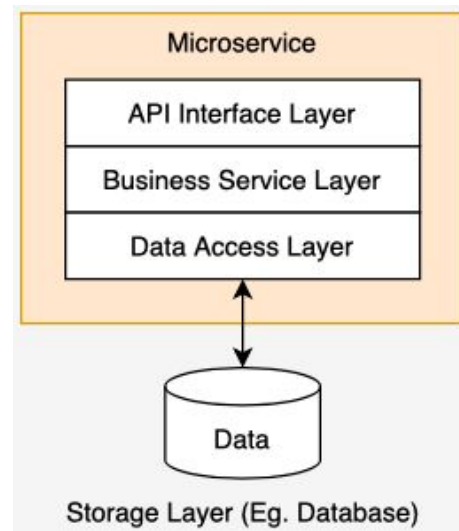
Capa de Acceso a Datos (DAL)

< > Capa de Acceso a Datos (DAL)

La capa de acceso a datos (**Data Access Layer**) es un componente de una arquitectura de software que es responsable de administrar el almacenamiento y la recuperación de datos de una aplicación.

Se encuentra entre la capa de lógica de negocios y el sistema de almacenamiento de datos.

El DAL está diseñado para ser reutilizable e independiente de la lógica comercial y la implementación del almacenamiento de datos. Se puede implementar utilizando diferentes tecnologías de acceso a datos y proporciona soporte para transacciones y otras funciones de acceso a datos.





Ejercicio N° 1

Cambios y Resultados

Singletoneando

Manos a la obra: 🙌

Vamos a modificar el código para agregar los objetos ResultSet correspondientes.

Te invitamos a que generes algunos registros en tu BD Wallet para poder probar las consultas.

Consigna: ✍️

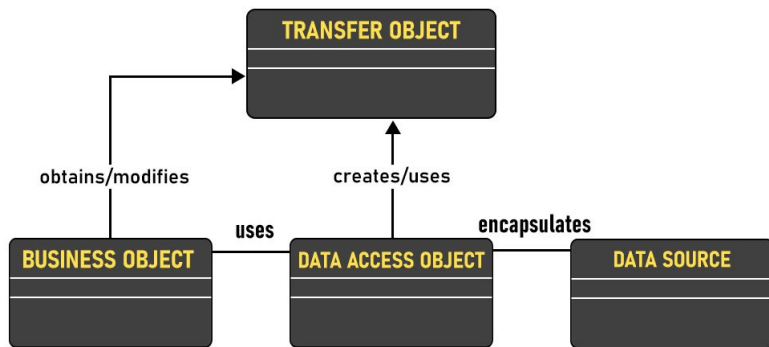
1. Agregar los objetos ResultSet para atrapar los resultados de las sentencias escritas.
2. Crear una tabla USUARIO con columnas “id, name, password, email, country”
3. Insertar cuatro registros de usuario.
4. Modificar el nombre de al menos dos usuarios.

Tiempo 🕒: 30 minutos

El patrón de diseño DAO

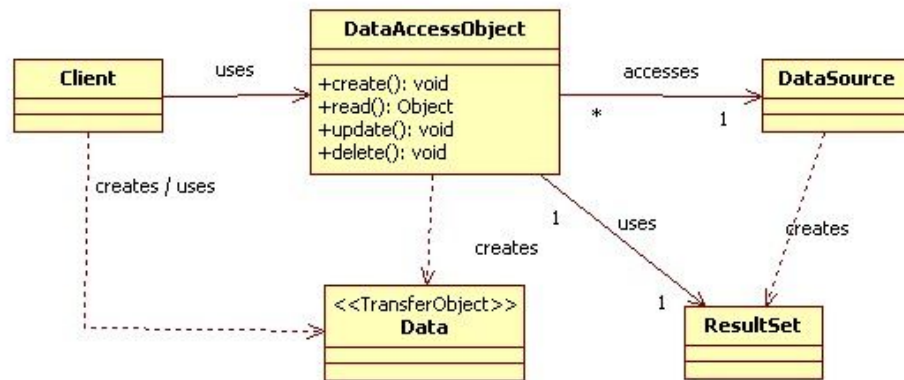
< > DAO

El patrón **Data Access Object** (DAO) pretende principalmente independizar la aplicación de la forma de acceder a la base de datos, o cualquier otro tipo de repositorio de datos. Para ello **se centraliza el código relativo al acceso al repositorio de datos en las clases llamadas DAO** . Fuera de las clases DAO no debe haber ningún tipo de código que acceda al repositorio de datos.



<> DAO

El patrón DAO propone separar por completo la lógica de negocio de la lógica para acceder a los datos, de esta forma, el DAO proporcionará los métodos necesarios para **insertar** , **actualizar** , **borrar** y **consultar (CRUD)** la información.



< > DAO

Esto lo vamos a lograr a través de cuatro clases:

- **Entidad** : va a ser la clase que va a representar a la tabla que queremos trabajar de la base de datos. Va tener como atributos las columnas de la tabla de la base de datos.
- **DAO**: esta clase va a ser la encargada de comunicarse con la base de datos, de conectarse con la base de datos, enviar las consultas y recuperar la información de la base de datos.
- **Servicio o Business Service** : va a tener toda la lógica de negocio del proyecto, usualmente se genera una para cada entidad. Es la que se encarga de obtener datos desde la base de datos utilizando el DAO y enviarla al cliente, o a su vez recibir la clase desde el cliente y enviar los datos al servidor, por lo general tiene todos los métodos CRUD (create, read, update y delete).



< > DAO

EntidadDaolmpl: esta clase va a extender de la clase DAO y se va encargar de generar las sentencias para enviar a la clase DAO, como un insert, select, etc. Y si estuviéramos haciendo un select, sería también, la encargada de recibir la información, que recupera la clase DAO de la base de datos sobre una entidad, para después enviarla al servicio, que será la encargada de imprimir dicha información.

Este es un objeto plano que implementa el patrón **Data Transfer Object (DTO)**, el cual sirve para transmitir la información entre el DAO y el Business Service.



El patrón DTO

DTO

Un DTO u objeto de transferencia de datos es simplemente una clase plana sin ningún tipo de lógica de negocio en su interior, sólo es usado para guardar datos dentro de él.

Por norma general los DTO son clases con atributos con la palabra clave final, es decir son clases inmutables. Además, suelen implementar la interfaz Serializable ya que pueden ser utilizados para convertirlos en bytes y enviar o obtener la información de alguna fuente.

```
1 public class UsuarioDTO implements Serializable {  
2     private final String nombre;  
3     private final String apellidos;  
4     // constructor y otros métodos  
5 }
```

DTO

Diferencias entre DTO y entidades

Los DTO y las entidades se utilizan comúnmente en aplicaciones Java, pero tienen diferentes propósitos.

Las **entidades** son **objetos que representan el estado de los datos en una base de datos** y normalmente se utilizan para implementar la **capa de persistencia** de una aplicación. Incluyen propiedades que se asignan a columnas en una tabla de base de datos, así como métodos para acceder y manipular los datos. Las entidades suelen estar diseñadas para ser persistentes, lo que significa que su estado puede almacenarse y recuperarse de una base de datos.

DTO

Diferencias entre DTO y entidades:

Los **DTO**, por otro lado, **se utilizan para la transferencia de datos entre capas** de una aplicación y no se preocupan por la persistencia. Están diseñados para ser simples y livianos, sin ninguna lógica de negocios, e incluyen sólo los datos necesarios para las necesidades específicas de cada capa. Los DTO se utilizan normalmente para transferir datos desde una capa de servicio a una capa de presentación o entre microservicios.

Implementando un DAO con métodos CRUD



Implementando CRUD

Veamos un ejemplo de clases DAO. Esto lo vamos a lograr a través de las cuatro clases vistas anteriormente:

1- **Entidad** : va a ser la clase que va a representar a la tabla que queremos trabajar de la base de datos.

```
public class Usuario {  
  
    private String nombre;  
    private String correoElectronico;  
    private Integer edad;  
}
```



Implementando CRUD

2- **DAO**: representa una capa de acceso a datos que oculta la fuente y los detalles técnicos para recuperar los datos. Es la clase donde tenemos los objetos de conexión, sentencia y resultado.

```
public class DAO {  
    //OBJETO CONNECTION ---> Encargado de INICIAR - TENER - MANTENER la conexion  
    protected Connection conexion = null;  
    //OBJETO RESULTSET ---> Guardar todos los datos que llegan de la DB (las filas de la consulta)  
    protected ResultSet resultado = null;  
    //OBJETO STATEMENT ---> "Tiene" las consultas... es donde generamos las sentencias a ejecutar!  
    protected Statement stmt = null;  
  
    private final String user = "root";  
    private final String password= "root";  
    private final String database = "nombre_database";  
  
    protected void conectarBase(){  
  
    protected void consultarBase(String sql) {  
  
    protected void desconectarBase(){  
  
}
```



Implementando CRUD

3- **EntidadDAO** : esta clase va a extender de la clase DAO y se va encargar de generar las sentencias SQL para enviar a la clase DAO. Este es un objeto plano que implementa el patrón Data Transfer Object (DTO), el cual sirve para transmitir la información entre el DAO y el Business Service.

```
public class UsuarioDAO extends DAO {  
    public List<Usuario> listarUsuario() throws Exception{  
        try {  
            //armado de la sentencia SQL  
            String sql = "SELECT nombre,correoElectronico,edad "  
                + "FROM Usuario; ";  
  
            //consultamos la DB haciendo uso del método del DAO  
            consultarBase(sql);  
  
            //Recorro el resultado de la consulta  
            Usuario usuario = null;  
            List<Usuario> usuarios = new ArrayList<>();  
  
            while(resultado.next()){  
                usuario = new Usuario();  
  
                usuario.setNombre(resultado.getString(2));  
                usuario.setCorreoElectronico(resultado.getString(3));  
                usuario.setEdad(resultado.getInt(4));  
  
                //lo agrego a la list!  
                usuarios.add(usuario);  
            }  
  
            return usuarios;  
        }  
    }  
}
```

Implementando CRUD

4- **Servicio o Business Service** : por lo general tiene todos los métodos CRUD (create, read, update y delete).

```
public class UsuarioService {  
    //llamado a UsuarioDAO  
    private UsuarioDAO dao;  
    // constructor para inicializarlo  
  
    public UsuarioService() {  
        this.dao = new UsuarioDAO();  
    }  
  
    public List<Usuario> listarUsuario() throws Exception{  
        try {  
  
            List<Usuario> usuarios = dao.listarUsuario();  
  
            return usuarios;  
        } catch (Exception e) {  
            throw new Exception("Error en listar usuario Servicio");  
        }  
    }  
}
```

LIVE CODING

¡A CRUDear!

Vamos a crear los métodos necesarios para crear, insertar, eliminar y modificar los datos de nuestra aplicación.

Para ello vamos a mejorar lo que veníamos haciendo, agregando las clases DAO necesarias e implementando lo aprendido.

Tiempo: 25 minutos



Ejercicio N° 1

CRUD

Singletoneando

Manos a la obra: 🙌

¡Finalmente vamos a unificar lo aprendido! Vas a trabajar sobre un nuevo proyecto donde organizarás las clases nuevamente para respetar el patrón DAO.

Consigna: ✍️

1. Crear las clases necesarias para interactuar con UserDAO, AccountDAO y CurrencyDAO .
2. Crear una nueva BD Wallet e insertar registros.
3. Modificar el nombre de al menos dos usuarios.

Tiempo 🕒: 30 minutos



Momento:

Time-out!



5 -10 min.



¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ **Comprender la ejecución de sentencias de modificación**
- ✓ **Aprender la implementación del objeto ResultSet.**
- ✓ **Conocer la utilización de la Capa de Acceso a Datos**
- ✓ **Comprender la implementación del patrón DAO**
- ✓ **Aprender el concepto de los DTOs.**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

